

Práctica guiada de laboratorio: construcción y validación de un entorno SDR

Esta práctica tiene como propósito que cada estudiante construya, configure y fortalezca su propio entorno de desarrollo para radio definida por software (SDR). La meta no es recibir un entorno preconfigurado, sino comprender la relación entre el sistema operativo, el gestor de entornos, las librerías, los drivers y el hardware de adquisición.

Objetivo general

Construir y validar un entorno de desarrollo reproducible para experimentación con SDR utilizando Conda, SoapySDR, Spyder y librerías asociadas al acceso de dispositivos RTL-SDR.

Resultados de aprendizaje

Al finalizar la práctica, el estudiante será capaz de instalar Miniconda, crear un entorno aislado con Python 3.10, instalar SoapySDR y módulos de hardware, integrar librerías para RTL-SDR, verificar el reconocimiento del dispositivo y diagnosticar problemas básicos de permisos o drivers.

Checkpoint 1. Instalación de Miniconda

Ejecuta en terminal los siguientes comandos:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
chmod +x Miniconda3-latest-Linux-x86_64.sh
./Miniconda3-latest-Linux-x86_64.sh
source ~/.bashrc
conda --version
```

Validación: el sistema debe responder mostrando la versión instalada de conda.

Checkpoint 2. Creación y activación del entorno de trabajo

```
conda create -n sdr_env python=3.10
conda activate sdr_env
python --version
```

Validación: verifica que la versión activa de Python sea 3.10 y que el prompt muestre el entorno `sdr_env`.

Checkpoint 3. Configuración de canales y prioridad

```
conda config --add channels conda-forge
conda config --set channel_priority strict
```

Validación: esta configuración reduce conflictos de dependencias y mejora la consistencia del entorno.

Checkpoint 4. Instalación de SoapySDR y módulos SDR

```
conda install soapy-sdr python-soapy-sdr
conda install soapysdr-tools
conda install soapy-rtl-sdr soapy-hackrf soapy-lms7
```

Validación: al concluir, el entorno debe contar con la capa de abstracción SoapySDR y con módulos para distintos dispositivos SDR.

Checkpoint 5. Instalación de dependencias del sistema

```
sudo apt update
sudo apt install -y build-essential cmake pkg-config libusb-1.0-0-dev libfftw3-dev git rtl-sdr
```

Validación: estas dependencias facilitan el acceso a hardware USB y utilitarios clásicos del ecosistema RTL.

Checkpoint 6. Configuración de permisos USB

```
sudo usermod -aG plugdev,dialout $USER
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Validación: cierra sesión y vuelve a entrar al sistema para que los cambios de grupos surtan efecto.

Checkpoint 7. Instalación explícita de la librería RTL-SDR dentro del entorno

Además de SoapySDR, conviene incorporar de forma explícita la librería base para RTL-SDR dentro del entorno Conda. Esto fortalece la compatibilidad y permite usar interfaces adicionales.

```
conda activate sdr_env
conda install -c conda-forge librtlsdr
```

Validación: la instalación de **librtlsdr** proporciona la librería base y archivos necesarios para acceso al hardware RTL2832U.

Checkpoint 8. Instalación de la librería Python rtlsdr

Para trabajar con el dispositivo desde una interfaz Python distinta de SoapySDR, instala también la librería **pyrtlsdr** dentro del mismo entorno.

```
conda activate sdr_env
pip install pyrtlsdr
```

Validación: esta etapa habilita una vía adicional de acceso programático al dispositivo RTL-SDR.

Checkpoint 9. Verificación de SoapySDR en terminal

```
SoapySDRUtil --info
SoapySDRUtil --find
SoapySDRUtil --probe="driver=rtlsdr"
```

Validación: el sistema debe listar drivers instalados y, si el dispositivo está conectado y con permisos correctos, debe reconocerlo.

Checkpoint 10. Verificación en Python con SoapySDR

```
import SoapySDR
from SoapySDR import Device

print(SoapySDR.__version__)
print(Device.enumerate())
```

Validación: si la salida muestra una lista de dispositivos o una estructura vacía sin errores, el binding de Python está correctamente instalado.

Checkpoint 11. Verificación en Python con rtlSdr

```
from rtlSdr import RtlSdr

sdr = RtlSdr()
print("Frecuencia central (Hz):", sdr.center_freq)
print("Tasa de muestreo (Hz):", sdr.sample_rate)
print("Ganancia (dB):", sdr.gain)
sdr.close()
```

Validación: si el script se ejecuta sin errores de importación o acceso al dispositivo, la librería **rtlSdr** quedó integrada correctamente al entorno.

Checkpoint 12. Integración opcional con Spyder

```
conda install spyder
conda install spyder-kernels
which python
```

Validación: si utilizas una instalación global de Spyder, coloca la ruta de Python del entorno como intérprete alternativo en la configuración de la consola IPython.

Problemas comunes

El dispositivo no aparece: verifica con **lsusb** si el sistema operativo lo detecta. Si aparece en **lsusb** pero no en SoapySDR, normalmente el problema está en permisos, reglas udev o drivers.

Error al importar rtlSdr: confirma que **librtlSdr** esté instalada en Conda y que **pyrtlSdr** se haya instalado dentro del entorno activo.

Conflictos de paquetes: crea un entorno limpio en lugar de mezclar instalaciones previas.

Conclusión

Un entorno SDR funcional no debe entenderse como un paquete ya hecho, sino como una construcción técnica que cada estudiante debe desarrollar y validar. La instalación de SoapySDR, junto con la librería base RTL-SDR y la interfaz Python rtlSdr, fortalece la interoperabilidad del entorno y mejora la preparación para prácticas posteriores de adquisición, PSD, espectrogramas y procesamiento de señales de RF.